Identifying LLMs from Prompts and Responses

CMPSC 448 Midterm Project Report

https://github.com/nicholash11/MidtermProject1CMPSC448

October 11 2026

Team leader: Nicholas Pham | Team members: Nicholas Pham, Conrad Cliff

Abstract

This project examines whether text classifiers can identify the large language model that generated a response and whether the user prompt improves that identification. The dataset contains 2,403 responses from GPT, Claude, and Llama, with one response from each model for each of 801 shared prompts. Prompt groups are kept within a single training, validation, or test split. Six recorded experiments compare convolutional neural networks (CNNs) and recurrent neural networks (RNNs) using the prompt alone, response alone, or both.

Response-only test accuracy reaches 82.09% for the CNN and 79.06% for the RNN. Prompt-only accuracy is 33.33% for both models. The highest recorded accuracy is 83.75% for the CNN using both fields, but a paired prompt-level analysis does not establish a reliable gain over response-only classification. The RNN falls to 65.56% when the prompt is included. Additional descriptive analysis finds substantial differences in response length and Markdown formatting; a formatting-only logistic regression baseline reaches 63.09%. These results support model attribution within this benchmark while leaving generalization to other tasks, model versions, and generation settings unresolved.

1 Problem definition

The task is supervised three-class authorship classification: given a prompt, a generated response, or their combination, predict GPT, Claude, or Llama. These labels refer to the three specific model versions in the dataset, rather than all possible members of each family. Classification accuracy measures identification of the generator; it does not measure response quality or whether a response is factually correct.

The required questions are RQ1, whether the response identifies its generator, and RQ2, whether the prompt adds useful information. RQ4 is investigated through length, lexical diversity, formatting, and simple feature-based classifiers. RQ3, generalization across tasks, is reserved for a future domain-held-out experiment because the recorded experiments do not provide that evaluation. The report follows the project brief and distinguishes recorded neural results from the additional analyses performed on the saved data.

2 Dataset collection and curation

The project uses published AlpacaEval model outputs collected on October 11, 2026, from revision cd543a149df89434d8a54582c0151c0b945c3d20. Each source has 805 rows. The code aligns prompts after lowercasing and collapsing whitespace. Four groups are removed, leaving 801. These pattern checks are a limited screening method rather than a comprehensive sensitive-content review.

Label	Generator version	Retained responses
GPT	gpt-4o-mini-2024-07-18	801
Claude	claude-3-sonnet-20240229	801
Llama	Meta-Llama-3-8B-Instruct	801

Every row contains LLM_name, LLM_Input, LLM_output, generator, prompt_id, and split. A row-level audit confirms 801 distinct prompt IDs and 801 distinct prompt texts. Every group contains exactly one row per label and identical prompt text across labels. No retained response is blank. Thus the dataset is exactly balanced, with 801 examples per model.

Prompt grouped splits

Split	Prompt groups	GPT	Claude	Llama	Total rows
Training	560	560	560	560	1680
Validation	120	120	120	120	360
Test	121	121	121	121	363
Total	801	801	801	801	2403

The split proportions are approximately 69.91%, 14.98%, and 15.11%. The code sorts retained normalized prompts, shuffles them with seed 42, allocates the first 70% to training and the next 15% to validation using integer truncation, and assigns the remainder to test. No prompt ID or exact prompt text crosses splits. Keeping all three responses to a prompt together avoids evaluating on prompts already used in training.

Duplicate and provenance checks

The code removes groups when a normalized response occurs under multiple distinct prompts; it permits identical responses from different generators to the same prompt. Five exact response strings each occur twice within training, producing 2,398 distinct outputs. All five pairs belong to the same prompt but different labels. None crosses splits. Zero repeated-output groups were removed, consistent with the filtering rule. Identical outputs remain inherently ambiguous for response-only classification.

The metadata records a SHA-256 digest for each source file. Those digests document provenance but were not revalidated against downloaded originals in this analysis. The AlpacaEval dataset page lists CC BY-NC 4.0, consistent with the license noted in the project metadata. Source attribution and the benchmark revision are retained in the references.

3 Models and experimental method

The six experiments compare a CNN and an LSTM classifier using prompt-only, response-only, and combined inputs. Both classifiers are implemented in PyTorch. The RNN label in the result tables refers to the LSTM implementation described below.

3 1 Text preprocessing and input representation

Text is converted to lowercase and tokenized with a regular expression that retains words, punctuation, and newline characters. Explicit model and organization names, including GPT, ChatGPT, Claude, Llama, OpenAI, Anthropic, and Meta AI, are replaced with the token “modelname.” This reduces identification through direct self-identification rather than other textual characteristics.

Prompt-only inputs retain the first 128 tokens; response-only inputs retain the first 256. Combined inputs concatenate up to 128 prompt tokens, a separator token, and up to 256 response tokens, giving a maximum length of 385. Separate token limits mean adding the prompt does not reduce the response’s 256-token allowance.

Each configuration has a vocabulary built only from training data: the 9,998 most frequent tokens plus padding and unknown tokens. Unseen tokens map to the unknown token, and shorter sequences are padded to the fixed input length. The recorded vocabulary contains 3,709 entries for prompt-only inputs and 10,000 for the other configurations.

3 2 Convolutional neural network

The CNN uses a trainable embedding layer with 64-dimensional token vectors and padding index zero. Three parallel one-dimensional convolutions use kernel widths of 3, 4, and 5, each with 64 output channels. ReLU activation follows each convolution. Activation positions are masked using the sequence’s non-padding length before global maximum pooling. The three pooled vectors are concatenated into a 192-dimensional representation. Dropout with probability 0.30 precedes a linear layer that produces three class logits for GPT, Claude, and Llama.

3 3 Recurrent neural network

The RNN uses a trainable 64-dimensional embedding layer and a single-layer, unidirectional LSTM with hidden size 64. The LSTM produces a hidden vector at each sequence position. Masked mean pooling averages these vectors over non-padding positions to form a 64-dimensional representation. Dropout with probability 0.30 is applied before a linear layer produces the three class logits.

Input configuration	Vocabulary size	CNN parameters	LSTM parameters
Prompt only	3,709	287,299	270,851
Response only	10,000	689,923	673,475
Prompt and response	10,000	689,923	673,475

3 4 Training procedure and checkpoint selection

Both models use the saved prompt-group split: 1,680 training examples, 360 validation examples, and 363 test examples. All three responses to a prompt stay in the same split. Training uses Adam with learning rate 0.002, cross-entropy loss, and batches of 32 examples. Training examples are shuffled each epoch. Gradients are clipped to a maximum global norm of 1.0 before each optimizer update.

The maximum training duration is 12 epochs. Validation loss is evaluated after every epoch. A checkpoint is saved only when validation loss improves by more than 0.0001 relative to the previously saved best loss. Training stops when three epochs have passed without an accepted improvement. The saved checkpoint is restored before test evaluation.

This threshold explains the prompt-only LSTM checkpoint: epoch 5 improves on epoch 2 by approximately 0.000064, less than the required 0.0001. Epoch 2 therefore remains selected. For the other five runs, the selected epoch also has the lowest recorded validation loss.

Python, NumPy, and PyTorch seeds are reset to 42 before each model is trained; the training DataLoader also uses a generator seeded with 42. The implementation enables deterministic PyTorch algorithms, uses four CPU threads, and trains on the CPU. Validation and test evaluation use batches of 64, disable gradient computation, and use evaluation mode so dropout is inactive.

The exported run environment is Python 3.12.14, PyTorch 2.14.1+cpu, NumPy 2.3.5, and Windows 11 build 26200. The recorded number of completed epochs and the selected checkpoint are shown below.

Model	Text used	Epochs completed	Selected epoch
CNN	Prompt	6	3
RNN	Prompt	5	2
CNN	Response	11	8
RNN	Response	11	8
CNN	Both	9	6
RNN	Both	6	3

3 5 Evaluation and confidence intervals

Predictions select the largest class logit. Evaluation reports accuracy, macro F1, per-class precision, recall and F1, and a confusion matrix. Matrix rows represent true labels and columns predicted labels, ordered GPT, Claude, Llama. The 363 saved predictions per run reproduce every recorded confusion matrix, accuracy, and macro F1. Overlapping metrics in the two JSON files agree.

The recorded 95% accuracy intervals use a prompt-group bootstrap with 2,000 resamples and seed 42. Correctness is averaged over the three responses to each test prompt. These prompt-level values are resampled with replacement, and the 2.5th and 97.5th percentiles of bootstrap accuracy form the interval. This preserves the shared-prompt structure. The recorded intervals are distinct from the additional 10,000-resample paired comparisons in Section 8.

4 Results for the required questions

Model	Text used	Correct of 363	Accuracy	Macro F1	Recorded 95% accuracy interval
CNN	Prompt	121	33.33%	0.2102	33.33 to 33.33%
RNN	Prompt	121	33.33%	0.3266	33.33 to 33.33%
CNN	Response	298	82.09%	0.8226	77.95 to 86.23%
RNN	Response	287	79.06%	0.7900	74.66 to 83.47%
CNN	Both	304	83.75%	0.8378	79.61 to 87.33%
RNN	Both	238	65.56%	0.6424	61.43 to 69.97%

The table reports the saved 95% accuracy intervals, computed using 2,000 prompt-group bootstrap resamples. Section 8 separately compares configurations using 10,000 paired prompt-group resamples.

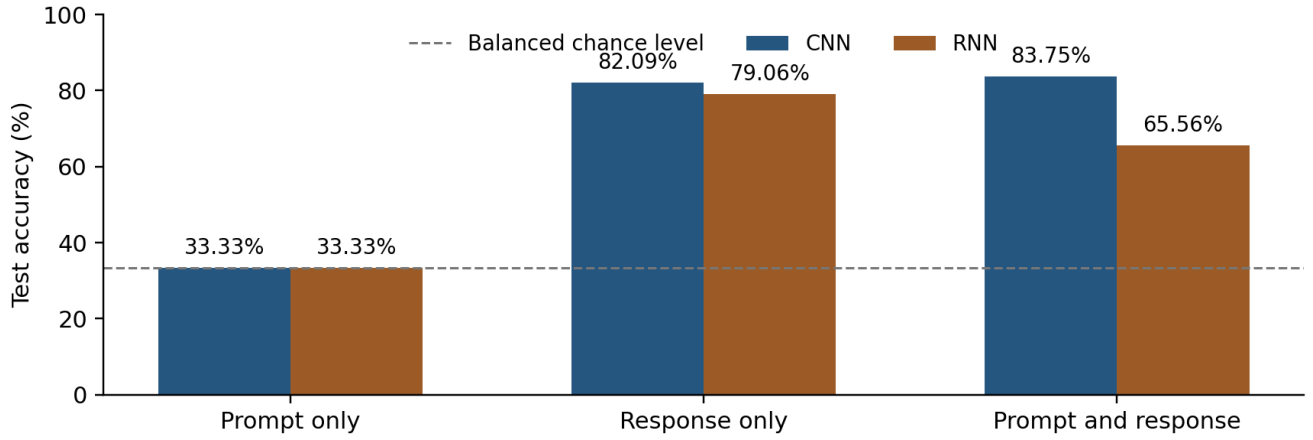


Figure 1 Test accuracy for all six recorded configurations. The dashed line is the 33.33% balanced chance level.

RQ1 Can responses identify their generator

Yes, within this held-out benchmark. The response-only CNN correctly identifies 298 of 363 responses, achieving 82.09% accuracy and 0.8226 macro F1. The response-only RNN identifies 287 correctly, with 79.06% accuracy and 0.7900 macro F1. Both are far above the 33.33% balanced chance level. The CNN has an observed 3.03 percentage-point advantage, although a paired uncertainty analysis does not establish that advantage reliably on this test set.

RQ2 Does the prompt help

Prompt-only classification reaches exactly 33.33% accuracy for both models. This is expected: a deterministic classifier must give the same prediction to the same prompt, and each test prompt appears once under each of the three labels. It therefore gets exactly one of the three rows correct. The result follows from the balanced shared-prompt construction, not simply from insufficient training.

Adding the prompt raises CNN accuracy by 1.65 percentage points, from 82.09% to 83.75%, or six additional correct rows. For the RNN, it reduces accuracy by 13.50 points, from 79.06% to 65.56%. Prompt inclusion therefore has no consistent benefit across these architectures. The CNN using both fields is the highest observed scorer; response-only CNN remains a simpler strong configuration.

5 Class performance and error patterns

Configuration	Class	Precision	Recall	F1
CNN response	GPT	0.9010	0.7521	0.8198
CNN response	Claude	0.7133	0.8843	0.7897
CNN response	Llama	0.8929	0.8264	0.8584
RNN response	GPT	0.8091	0.7355	0.7706
RNN response	Claude	0.8230	0.7686	0.7949
RNN response	Llama	0.7500	0.8678	0.8046
CNN both	GPT	0.8571	0.7934	0.8240
CNN both	Claude	0.7754	0.8843	0.8263
CNN both	Llama	0.8938	0.8347	0.8632

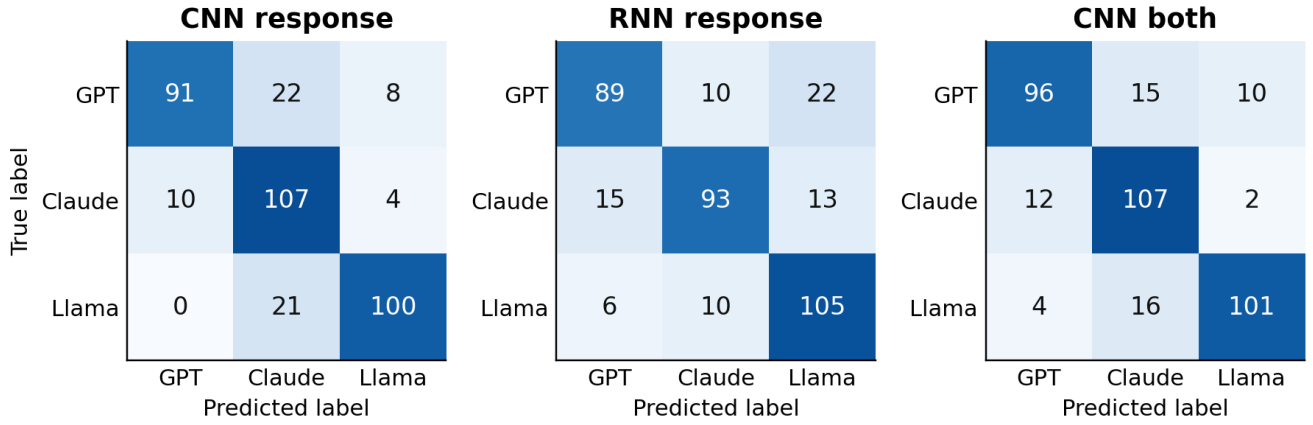


Figure 2 Test confusion matrices. Each true class contains 121 responses; darker diagonal cells indicate correct classifications.

The response-only CNN is most precise when it predicts GPT (90.10%), but GPT recall is only 75.21%. Of 30 GPT errors, 22 are predicted as Claude and eight as Llama. Claude has the highest recall (88.43%) but the lowest precision (71.33%): the model predicts Claude 150 times, including 43 non-Claude responses. Llama has the highest F1 (0.8584).

With both fields, the CNN improves GPT recall to 79.34% and Llama recall to 83.47%, while Claude recall stays at 88.43%. Its errors fall from 65 to 59. The improvement in macro F1 from 0.8226 to 0.8378 reflects a modest improvement across the class F1 scores rather than a gain limited to one dominant class.

The response-only RNN has its highest recall for Llama (86.78%). In the prompt-and-response RNN, Llama recall falls to 41.32% and F1 to 0.5025: 48 of 121 Llama responses are predicted as GPT and 23 as Claude. This accounts for much of that configuration's performance loss and shows why aggregate accuracy should be accompanied by class-level analysis.

6 Training behavior and model selection

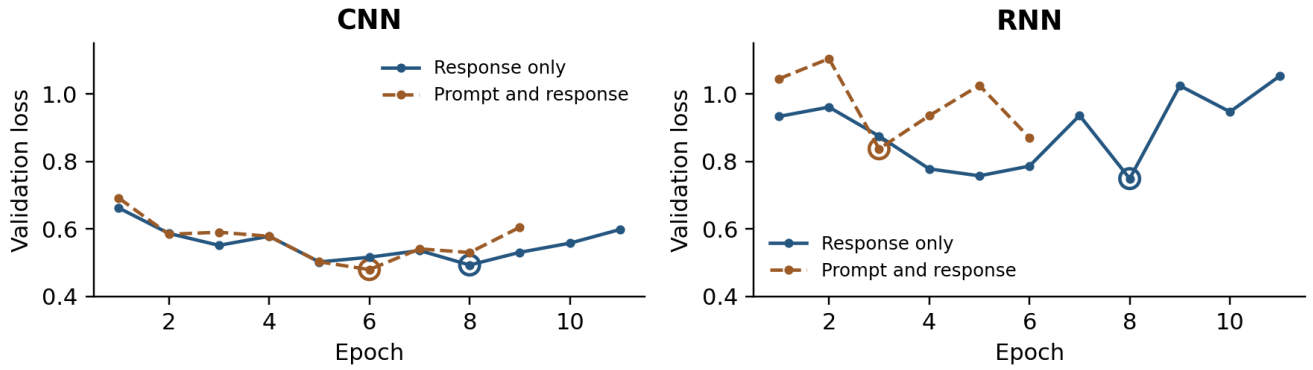


Figure 3 Recorded validation loss across epochs. Open circles mark the selected checkpoints for the response-only and combined-input runs.

Configuration	Selected epoch	Training loss at selection	Validation loss at selection	Recorded seconds
CNN response	8	0.2384	0.4930	16.01
RNN response	8	0.3175	0.7489	15.76
CNN both	6	0.3859	0.4795	17.72
RNN both	3	0.8270	0.8372	12.80

Validation loss and overfitting

For the response-only CNN, training loss decreases from 1.0275 to 0.1593 over 11 epochs. Validation loss reaches its minimum of 0.4930 at epoch 8 and rises to 0.5982 by epoch 11. The widening gap is consistent with overfitting after the selected checkpoint. The response-only RNN shows a similar pattern: training loss falls to 0.1468, while validation loss rises from its epoch-8 minimum of 0.7489 to 1.0536 at epoch 11.

The combined-input CNN reaches its lowest validation loss at epoch 6 (0.4795). The combined-input RNN reaches its lowest loss at epoch 3 (0.8372), even though validation accuracy rises from 62.22% at epoch 3 to 71.11% at epoch 6. Selecting by loss and selecting by accuracy can produce different checkpoints. There are no saved epoch-6 test predictions, so the effect of an alternative checkpoint cannot be quantified without another evaluation.

Prompt only training and the checkpoint discrepancy

Prompt-only validation accuracy remains 33.33% at every recorded epoch for both architectures. The prompt-only RNN selects epoch 2 with validation loss 1.098756, while epoch 5 records 1.098693. This improvement of approximately 0.000064 is below the code's required 0.0001 minimum improvement. The checkpoint therefore remains at epoch 2, and training stops after three epochs without an accepted improvement.

Timing interpretation

Elapsed values range from 3.12 to 17.72 seconds. Timing starts before the training loop and includes checkpoint restoration, test evaluation, and the bootstrap calculation. Without CPU specifications or repeated measurements, it cannot support a general speed ranking.

7 RQ4 Characteristics that distinguish the outputs

Additional analysis measures eight response features: word count, character count, type-token ratio, line count, list-item lines, bold markers, code-fence markers, and heading lines. Words are lowercase Unicode word sequences matched with a word-boundary regex; type-token ratio is distinct words divided by total words. List lines begin with a bullet or numbered marker, heading lines begin with one to six hash marks, and bold and code markers count literal double asterisks and triple backticks. These operational definitions count text patterns, not fully parsed Markdown.

Response statistic	GPT	Claude	Llama
Mean word count	283.9	223.2	305.8
Median word count	282	212	321
Mean type-token ratio	0.576	0.588	0.512
Mean line count	26.4	19.3	23.7
Responses with list items	64.42%	65.17%	66.92%
Responses with bold markers	64.67%	4.87%	54.68%
Responses with code fences	7.99%	6.87%	6.99%

Statistics above use all 801 responses per class and are descriptive. Claude responses are shorter on average and rarely contain bold markers (4.87%), compared with GPT (64.67%) and Llama (54.68%). Llama has the lowest mean type-token ratio, but this measure is length-dependent and should not be interpreted as an independent measure of vocabulary richness. List and code-fence prevalence are similar across labels.

Additional feature based classification experiment

Feature set	Test accuracy	Macro F1
Length only	47.38%	0.4712
Formatting only	63.09%	0.6175
Lexical diversity only	43.53%	0.3916
All descriptive features	63.36%	0.6307

These new baselines use the original split: 1,680 training rows and 363 test rows. A StandardScaler and logistic regression are fitted only on training data, using fixed C = 1, maximum iterations = 2,000, and seed 42. No test-based tuning is performed. The analysis uses scikit-learn 1.8.0; it does not retrain the recorded neural models.

Formatting-only features reach 63.09%, compared with 47.38% for length alone and 43.53% for type-token ratio alone. All eight features reach 63.36%. Thus shallow structural signals are predictive in this dataset, although the response-only CNN still performs 18.73 percentage points better than the full feature baseline. These comparisons do not prove which features the CNN or RNN uses. Testing that claim requires removing formatting or controlling length and retraining the neural classifiers.

8 Uncertainty and limits of the findings

The saved test predictions permit paired comparisons because each run classifies the same 363 responses. To account for shared prompts, an additional bootstrap resamples the 121 complete test prompt groups with replacement 10,000 times, using seed 42. Each sampled group contributes all three model responses. The table reports percentile 95% intervals for the accuracy difference; these intervals measure uncertainty across held-out prompt groups, not variation from retraining with different random seeds.

Comparison	Accuracy difference	Prompt bootstrap 95% interval	Newly correct / newly wrong
CNN both minus CNN response	+1.65 points	-0.83 to +4.13 points	15 / 9
CNN response minus RNN response	+3.03 points	-0.55 to +6.89 points	37 / 26
RNN both minus RNN response	-13.50 points	-18.18 to -8.81 points	29 / 78

The CNN's gain from adding prompts has an interval from −0.83 to +4.13 percentage points, including zero. Its response-only advantage over the RNN also includes zero. Both are observed differences worth retesting, rather than established improvements. The RNN's combined-input loss has an interval from −18.18 to −8.81 points, consistently below zero for the fixed saved predictions.

Sources of uncertainty

Only one model version represents each family and one training seed is recorded. Different model versions, system prompts, decoding settings, or response templates may change the distinguishing features. Neural architecture and training are documented in Section 3, but the source generators' system prompts and decoding settings are not specified in the project files. Results therefore describe these specific benchmark generators under this split.

Grouped splitting blocks exact prompt reuse across splits, but it does not rule out semantically similar prompts or task overlap. Public benchmark text may also differ from real-world text that has been edited, shortened, paraphrased, or stripped of formatting. The RQ4 analysis makes formatting a credible dataset signal; it does not establish a stable, model-intrinsic fingerprint.

RQ3 Generalization across tasks

No domain-held-out experiment is recorded, so this report does not claim that attribution generalizes across tasks. A follow-up should assign prompt groups to documented task categories, train on several categories, and evaluate on a disjoint category while preserving model balance. Both neural classifiers should be evaluated with response-only and combined inputs, with class-level metrics and prompt-group uncertainty. This would distinguish performance on unseen prompts from performance on unseen domains.

Priority follow up experiments

Repeat training with multiple seeds and compare prompt-group intervals across runs. Then remove Markdown markers, match response lengths, and retrain to measure reliance on structural signals. For combined-input RNNs, test the effects of the longer sequence, prompt-first ordering, and mean pooling. Both input modes retain up to 256 response tokens, so the combined-input loss cannot be explained by a smaller response token allowance.

9 Lessons and conclusion

The strongest methodological lesson is that dataset construction determines what a baseline can mean. Because every prompt appears under every label, chance-level prompt-only accuracy is an expected control. Prompt-group splitting supports a more credible assessment on unseen prompts, while class-level metrics reveal failures that aggregate accuracy conceals.

More input does not automatically improve a classifier. The combined-input CNN yields a small observed gain, whereas the combined-input RNN loses substantial accuracy, especially on Llama. Training curves also show that validation loss and validation accuracy can select different epochs. Checkpoint selection, sequence handling, and input representation therefore need explicit documentation.

The additional RQ4 experiment demonstrates that response formatting contains useful attribution information. That is a reason to test robustness under formatting changes, rather than assume the classifiers have learned a universal linguistic signature. Paired prompt-level uncertainty further prevents a small test-set difference from being overstated.

The project answers RQ1 positively within this benchmark: response-only CNN and RNN accuracy reaches 82.09% and 79.06%. RQ2 shows that prompts alone provide no label information under this balanced design and that including prompts has architecture-dependent effects. The highest recorded configuration is CNN with prompt and response at 83.75%, but its improvement over response-only CNN remains uncertain. Domain transfer, formatting ablations, and repeated-seed training are the next steps needed to establish robustness.

References

[1] Tatsu Lab. AlpacaEval published model outputs and dataset license page. https://github.com/tatsu-lab/alpaca_eval and https://huggingface.co/datasets/tatsu-lab/alpaca_eval. Dataset license listing accessed October 11, 2026.

Pinned source revision: cd543a149df89434d8a54582c0151c0b945c3d20. The project metadata specifies the following paths within that revision:

results/gpt-4o-mini-2024-07-18/model_outputs.json

results/claude-3-sonnet-20240229/model_outputs.json

results/Meta-Llama-3-8B-Instruct/model_outputs.json